Bysykler i Bergen prosjekt-rapport

Brage Nybakk INF161

Innhold

1. Datatilberedning og datautforskning 2
1.1 Første overblikk 2
1.2 Første resampling, velger en felles tidsperiode 2
1.3 Første visualiseringer med analyse 3
1.4 Resampling nummer 2. Sletting av data-hull 6
1.5 Nye visualiseringer med ferdig merget data 7
2. Variabelutvinning 8
2.1 Henter ut informasjon fra dato 8
2.2 «One-hot» encoding 9
2.3 Trigonometrisk encoding hos sykliske verdier 9
3. Modellering 10
3.1 Trener baseline modeller 10
3.2 Trener ulike reg. modeller og sjekker nøyaktighet 10
3.3 Sjekker generaliseringsevnen 11
3.4 Visualiseringer av modellene 12
3.5 Tester ulike hyperparametere av KNN 13
3.6 Prediksjon på test-data 13
3.7 Prediksjons-resultat 13
4. Konklusjon 14
4.1 Forbedringer 14
4.2 Utfordringer underveis 15
4.3 Refleksjon 15
Kilder 15
Bruk av KI 16

Introduksjon – Kort om prosjektet

Data science prosjekt der oppgaven er å lage en modell, som beregner antall ledige sykler den neste timen på et gitt antall stasjoner i Bergen sentrum. Det er gitt tre litt ulike sett rådata. Disse skal analyseres og visualiseres, og settes sammen til ett felles datasett som min utvalgte modell skal trenes på. Innleveringen består av filene; «train.py», som er selve piplinen til datasettet. Her går rådataen til en felles csv-fil, samt modellvalg og evaluering, og trening av min modell. «predict.py» som bruker den lagrede modellen til å lage prediksjoner for neste hele klokketime. Og denne rapporten, samt en hvordan_bruke.txt.

I denne rapporten vil jeg grundig, men konsist gå gjennom alle stegene jeg gjorde for å ende opp med min «train.py» og min utvalgte maskinlæringsmodell. Alt jeg skriver jeg har gjort viser til alt du kan finne i «train.py», ellers er alle figurer hentet fra visualiseringer jeg har gjort på min data-analyse. Mye fra første innlevering av prosjektet.

1. Datatilberedning og datautforskning

1.1 Første overblikk

Laster inn nødvendige pakker og leser inn dataen. Tar et raskt overblikk over kolonnene, de mulige «featuresa» på datasettene. Ser at datasettet «stations.csv» og «trips.csv» inneholder data på bredde og lengdegradene til de ulike stasjonene. Ser at starten på hvert av datasettene er forskjellig.

1.2 Første resampling, velger en felles tidsperiode

Når vi jobber med tidsbestemt data, som skal noe i fremtiden er det veldig viktig å unngå data-lekasjer. Tiden til hver av datasettene at de strekker seg over forskjellige perioder. Om jeg skulle splittet rådataen for visualisering før merging senere risikerer jeg å se hvordan test-dataen ser ut og dermed «se inn i fremtiden». Derfor er jeg nødt til å først finne en felles overlapp, og klippe dataen slik at de strekker seg over en felles periode.

For å få all dataen etter time for time fra start blir jeg nødt til å resample. Dette gjør jeg på ulike måter med de ulike settene.

For «stations.csv» bruker jeg et LOCF-prinsipp, der observasjon ved forrige time blir brukt helt frem til neste. Fra «trips.csv» utnytter jeg kun de turene som er inn og ut av mine utvalgte stasjoner, og jeg resampler til trips_in og trips_out kolonner tilhørende hver av stasjonene.

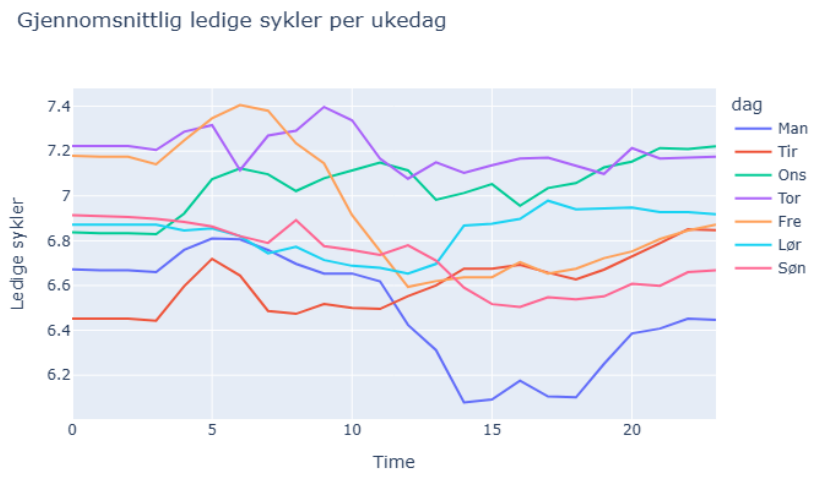
For «trips.csv» resamplet jeg også etter hver hele time. Men da kunne tomme verdier dukke opp etter som de ikke nødvendigvis alltid er turer inn eller ut. Disse satte jeg bare til 0, siden dette er aggregering, og ikke amputering av manglende verdier. For «weather.csv» valgte jeg å bruke gjennomsnittet av alle målinger til de tomme verdiene når jeg resamplet. På dette tidspunktet har jeg også gjort alle timestamp elementer til datetime objekt som gjør de lettere å håndtere.

Som nevnt tidligere er det data om bredde og lengdegradene til stasjonene. Dette er ikke data som hjelper å predikere ledige sykler, så jeg tolker det som unødvendige data og velger å slette disse kolonnene fra settene, etter som dette kan være støy i opptreningen av modeller. I tillegg sletter jeg «skipped uppdates» fra «stations.csv» før jeg sorterer dataen kronologisk i train-, val- og testdata så jeg er helt sikker på at jeg unngår data-lekkasjer.

Jeg velger å splitte dataen i 70/15/15 format. Dette gir en størst andel i treningsdata, som er viktig for å trene en god modell. I tillegg til en greit fordelt mengde der jeg kan sjekke nøyaktigheten til modellene.

1.3 Første visualiseringer med analyse

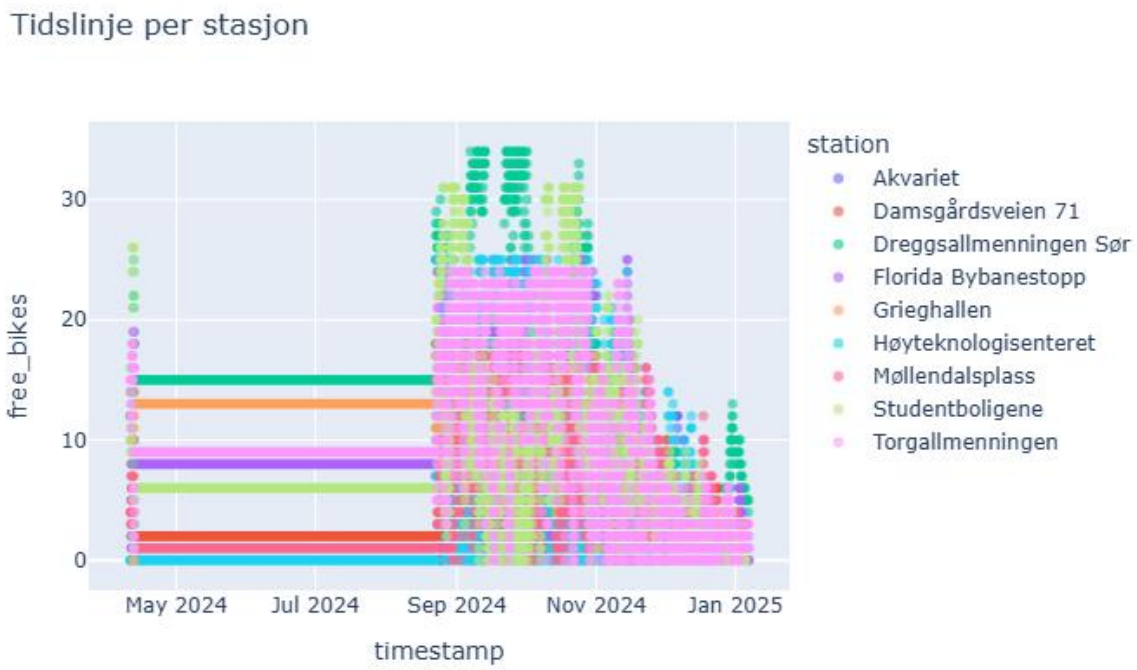
Visualiserer «stations.csv»:



Figur 1.1, strekdiagram over gjennomsnittlig ledige sykler per ukedag

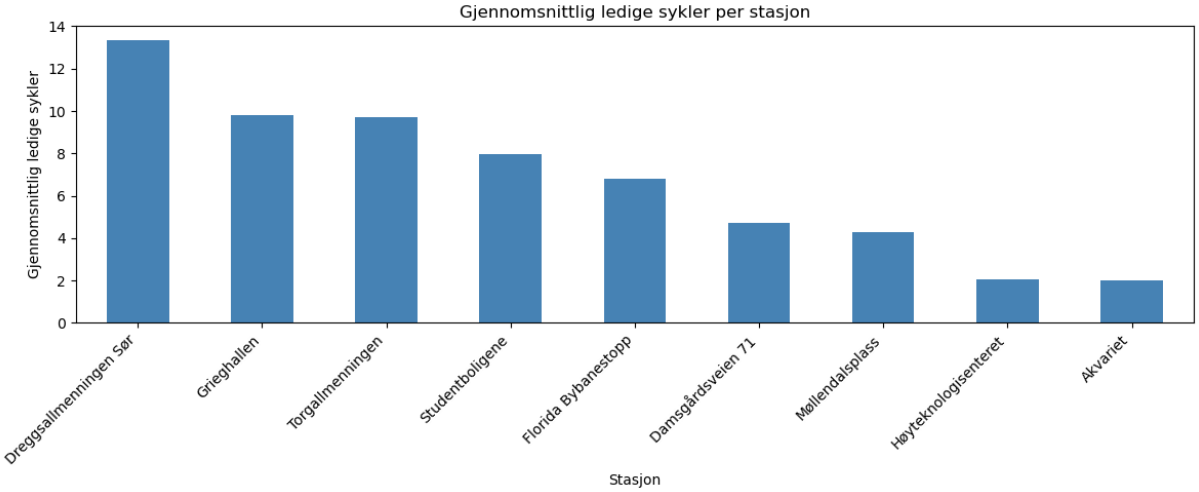
På første figur kan man klart se at det er en sammenheng mellom hvilken ukedag det er, og antall ledige sykler når på dagen i gjennomsnitt. F.eks. i på mandager ser det ut til å være færre ledige sykler på ettermiddagen. Dette er noe som vil være nyttig for modellene jeg

velger å teste senere. Jeg har hentet ut ukedag fra datetime objektet som nå er under timestamp kolonnen.



Figur 1.2, scatterplot, observasjoner over tid

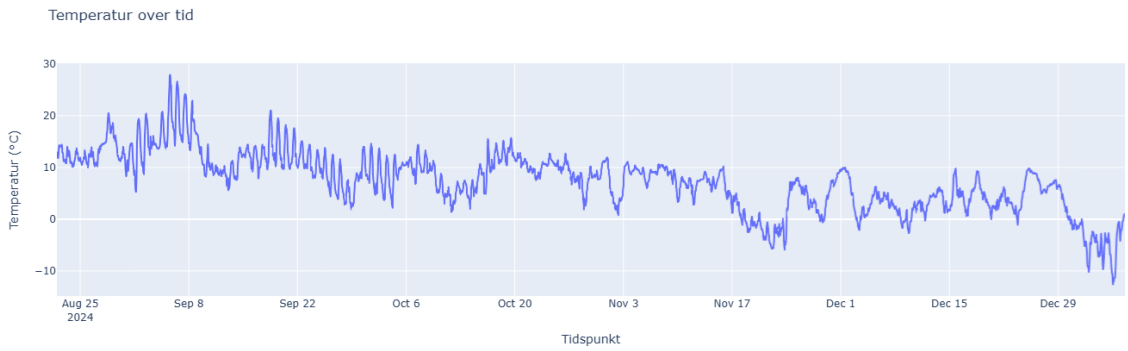
I figur 1.2 har det skjedd noe interessant. Dette er en scatter-figur over tiden til all dataen og antall ledige sykler. Hvert punkt er en observasjon og fargene representerer hvilken stasjon det er. Viktig å merke seg at alt dette er data *etter* jeg har resamplet. Det interessante her er at i perioden fra april til slutten av august er det ingen nye observasjoner. Min første tanke er at her er det noe jeg har oversett, og dette er noe jeg må se nærmere på.



Figur 1.3, stolpediagram, snitt av ledige sykler per stasjon

Figur 1.3 viser oss antallet ledige sykler over alle de aktuelle stasjonene i snitt. Dette kan si noe om antall plasser, men også populariteten på hvor det er vanligst å sykle fra.

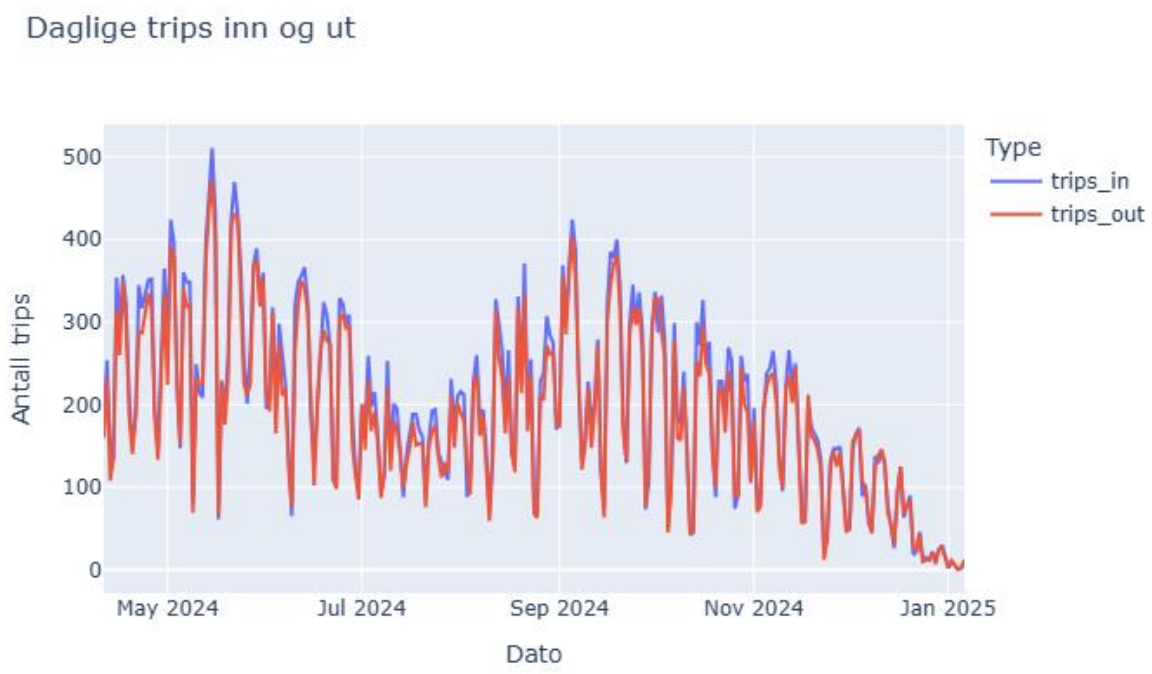
Visualiserer «weather.csv»:



Figur 1.4, tidslinje av temperatur

Enkel visualisering av vær-dataen. Ser hvordan temperaturen svinger, men synker mer mot vinteren. Viktig å merke at dataen er resamplet etter totale gjennomsnitt, som kan være grunnet til at det svinger såpass mye.

Visualiserer «trips.csv»:

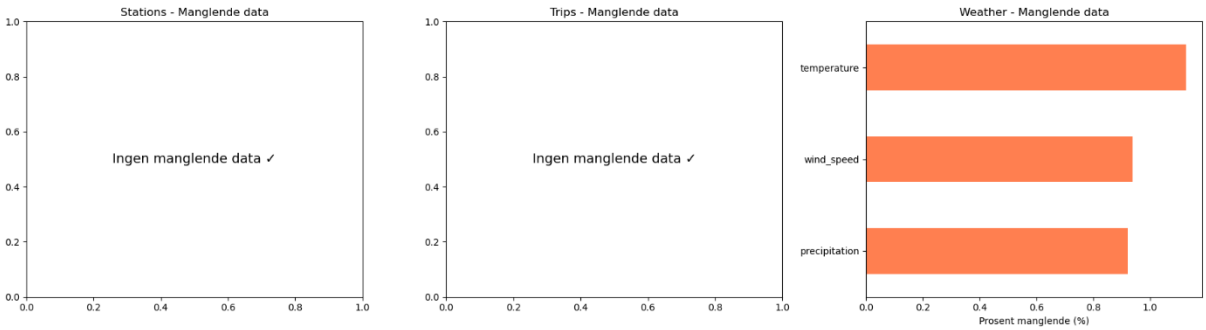


Figur 1.5, tidslinje over reiser inn og ut

Tidslinje over trips inn og ut. Ser at det er en sterk korrelasjon mellom trips_in og trips_out, men at turer inn kan overgå turer ut litt. Etter som dette er fra et utvalg av bysykkel-stasjoner kan det si noe om at dette er mer populære stasjoner å parkere sykkelen på enn å kjøre ut fra.

1.4 Resampling nummer 2. Sletting av data-hull

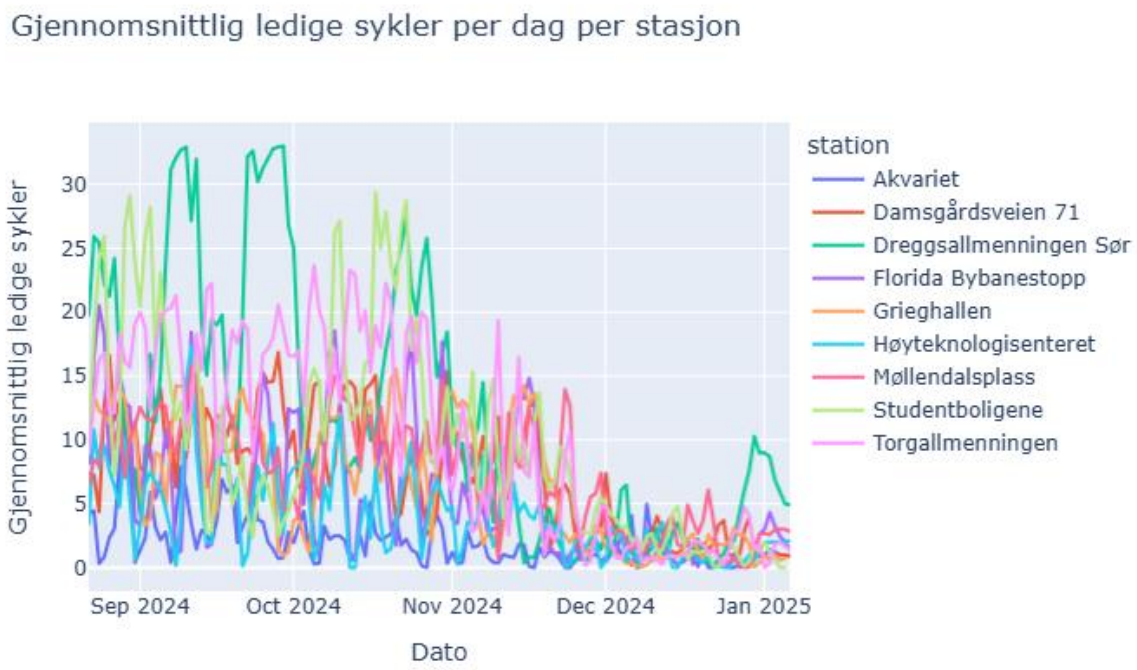
Etter å ha sett på dataen gjennom visualiseringer, blir jeg usikker på om jeg har gått glipp av manglende data. Dermed går jeg tilbake i dataen og lager jeg en figur for å sjekke dette.



Figur 1.6, tre stolpediagram over manglende data per datasett

Denne figuren viser at det er kun i «weather.csv» som det befinner seg «NaN» verdier, som jeg allerede har tatt hånd om. Dette tilsier at det rett og slett er et hull i tidslinjen mellom april og august. Jeg tenkte først det kunne være en slags pilotperiode, men trips-filen har ikke et lignende hull, så jeg antar at det har vært noe feil med innhentingen av data i denne perioden.

Jeg velger å løse problemet med å merge dataen sammen og slette perioden fra april til 22. august, som er første registrering etter hullet. Jeg prøver å visualisere figur 1.2 på nytt i ett litt annet format for å se om jeg oppdager noe lignende igjen. Til slutt legger jeg til «target-verdien» i dataen, nemlig ledige sykler neste time eller «fb_next_h», som er det jeg skal predikere.

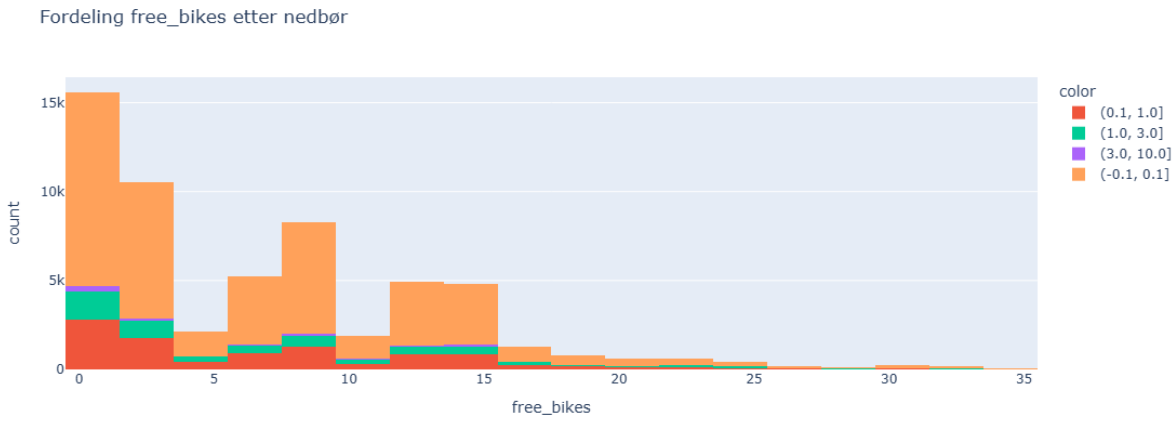


Figur 1.7, tidslinje over gjennomsnittlig ledig sykler

Figur 1.7 viser ingen nye datahull og dataen ser nå ut til å være kontinuerlig over tidsperioden.

1.5 Nye visualiseringer med ferdig merget data

Nå har jeg fått dataen samlet i en dataframe og valgt ut de featuresa jeg tror kommer til å være nyttige for videre variabelutvinning. Prøver å visualisere for å se nye sammenhenger på tvers av datasettene.



Figur 1.8, fordeling ledig sykler med mengde nedbør

Her kan man se fordeling av ledige sykler etter mengde nedbør. Sammenhengen her var mindre enn forventet. Konklusjonen her, er at det regner ofte i Bergen, men det regner ofte lite og dette påvirker ikke ledige sykler i så stor grad.

2. Variabelutvinning

Neste steg i data science-prosessen er å utvinne nyttige features fra den dataen jeg allerede har. På dette tidspunktet ser min tilgjengelige data slik ut:

```
Train data sample:
                station                timestamp  free_bikes  free_spots  trips_out  trips_in  temperature  precipitation  wind_speed  fb_next_h
0               Akvariet 2024-08-22 00:00:00+00:00         2.0        16.0        0.0       0.0         12.6            0.0        22.4        2.0
1        Studentboligene 2024-08-22 00:00:00+00:00         6.0        25.0        0.0       0.0         12.6            0.0        22.4        6.0
2  Dreggsallmenningen Sør 2024-08-22 00:00:00+00:00        15.0        19.0        0.0       0.0         12.6            0.0        22.4       15.0
3         Møllendalsplass 2024-08-22 00:00:00+00:00         1.0        16.0        0.0       0.0         12.6            0.0        22.4        1.0
4     Florida Bybanestopp 2024-08-22 00:00:00+00:00         8.0        17.0        0.0       0.0         12.6            0.0        22.4        8.0
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

2.1 Henter ut informasjon fra dato

Ved bruk av datetime objektet som tiden nå er satt til kan man hente ut informasjon om uke-dag og måned. I tillegg velger jeg å legge til en fb_prev_h feature som kan være nyttig for opptrening av modellen senere. Nedsiden er at dette gjør at jeg får en NaN-verdi siden første registrering ikke har den dataen. Velger å slette disse radene. Nå ser dataen slik ut, sorter bare på tid:

```
Train data sample:
                   station                timestamp  free_bikes  free_spots  trips_out  ...  wind_speed  fb_next_h  hour  weekday  fb_prev_h
1                  Akvariet 2024-08-22 01:00:00+00:00         2.0        16.0        0.0  ...        25.3        2.0     1        3        2.0
9900     Florida Bybanestopp 2024-08-22 01:00:00+00:00         8.0        17.0        0.0  ...        25.3        8.0     1        3        8.0
16476  Høyteknologisenteret 2024-08-22 01:00:00+00:00         0.0        25.0        0.0  ...        25.3        0.0     1        3        0.0
13204           Grieghallen 2024-08-22 01:00:00+00:00        13.0         2.0        0.0  ...        25.3       13.0     1        3       13.0
26343      Torgallmenningen 2024-08-22 01:00:00+00:00         9.0        15.0        0.0  ...        25.3        9.0     1        3        9.0

[5 rows x 13 columns]
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

2.2 «One-hot» encoding

For å utnytte data om hvilken stasjon det er bruker jeg one-hot encoding. Siden modellene bare kan trenes på numeriske data lager jeg dummie variabler av alle stasjonene. Det vil si at hver stasjon får en true eller false verdi, som tilsvarer 1 eller 0 i minne, som sier hvor observasjonen hører til. Dette skaper 8 ekstra kolonner. Man kan droppe en på grunn av 8 false vil si at den siste stasjonen er true. Vi har 8 nye kolonner og station er fjernet. Nå ser dataen slik ut:

```
Train data sample:
                    timestamp  free_bikes  free_spots  ...  station_Møllendalsplass  station_Studentboligene  station_Torgallmenningen
0     2024-08-22 01:00:00+00:00         2.0        16.0  ...                        0                        0                         0
9893  2024-08-22 01:00:00+00:00         8.0        17.0  ...                        0                        0                         0
16465 2024-08-22 01:00:00+00:00         0.0        25.0  ...                        0                        0                         0
13195 2024-08-22 01:00:00+00:00        13.0         2.0  ...                        0                        0                         0
26326 2024-08-22 01:00:00+00:00         9.0        15.0  ...                        0                        0                         1

[5 rows x 20 columns]
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

2.3 Trigonometrisk encoding hos sykliske verdier

For de tidsbestemte verdiene går de i sykluser. F.eks. går timene fra 0 til 23. Problemet her kan være at 0 og 23 kan se ut som langt fra hverandre for modellen, mens det er nærme i teorien. Dette kan gjøres noe med ved å projisere verdien i en sirkel, slik at starten og slutten møtes jevnt. I tillegg velger jeg å legge til måned og dag. Så gjør jeg alle disse sykliske tidsbestemte til sin- og cos-verdier, og sletter de vanlige. Dermed blir det endelige datasettet seende slik ut, der alle kolonnen er features bortsett fra fb_next_h:

```
=== ENDELIGE DATASETT ===
Train: 29627 rader, 2024-08-22 01:00:00+00:00 til 2025-01-06 14:00:00+00:00
Val: 12273 rader, 2025-01-06 16:00:00+00:00 til 2025-03-05 14:00:00+00:00
Test: 12508 rader, 2025-03-05 16:00:00+00:00 til 2025-05-02 14:00:00+00:00

Train data sample:
Index(['timestamp', 'free_bikes', 'free_spots', 'trips_out', 'trips_in',
       'temperature', 'precipitation', 'wind_speed', 'fb_next_h', 'hour_sin',
       'hour_cos', 'weekday_sin', 'weekday_cos', 'month_sin', 'month_cos',
       'day_sin', 'day_cos', 'fb_prev_h', 'station_Damsgårdsveien 71',
       'station_Dreggsallmenningen Sør', 'station_Florida Bybanestopp',
       'station_Grieghallen', 'station_Høyteknologisenteret',
       'station_Møllendalsplass', 'station_Studentboligene',
       'station_Torgallmenningen'],
      dtype='object')
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

3. Modellering

Neste steg er å trene ulike modeller og teste nøyaktigheten mot valideringsdataen. Som nevnt vil target verdien, altså «y», være ledige sykler neste time «fb_next_h». Og resten vil være features, altså X. Viktig å påpeke at siden prediksjonen er en kontinuerlig verdi, altså et tall vil modellene jeg bruker være regresjonsmodeller. Alle modeller trenes kun på treningsdata, men for å sjekke nøyaktighet skal modellene predikere «y» mot valideringsdata, og sjekke RMSE mot den ekte y-verdien til valideringsdataen. Dette tallet sier noe om avstanden mellom predikasjonen og den faktiske verdien, så den vil vi ha så lav som mulig.

N – antall observasjoner

x – inndata

y – faktiske observasjoner

f(x) – modellens predikert verdi

$$RMSE = \sqrt{\sum_{i=1}^{N} f(x_i) - y_i{}^2}.$$

3.1 Trener baseline modeller

For å sette en grunnlinje for hvor god modellen jeg trener må være, trener jeg to grunnlinjemodeller som baserer seg på gjennomsnitt og median. For alle modellene mine bruker jeg sklearn bibloteket, slik som for DummyRegressor:

```
Dummy_mean : med en RMSE på 6.078
Dummy_median : med en RMSE på 4.068
```

Dette er avrundet verdi. Dette betyr at regresjonsmodellene jeg trener må ha en RMSE bedre (lavere) enn 4.

3.2 Trener ulike reg. modeller og sjekker nøyaktighet

Nå er vi kommet til hoved-attraksjonen i data science prosessen, nemlig å trene maskinlæringsmodellene på all dataen vi har forberedt. Jeg har valgt ut 7 ulike modeller jeg har lyst til å trene dataen min på, og satte noen tilfeldig, men passende hyperparameter der det var behov. Videre sjekker jeg slik som grunnlinje-modellene:

```
Trener Linear...
Linear: RMSE = 0.740
Trener RandomForest...
RandomForest: RMSE = 0.782
Trener GradientBoosting...
GradientBoosting: RMSE = 0.738
Trener KNN...
KNN: RMSE = 0.879
Trener SVR...
SVR: RMSE = 0.727
Trener Lasso...
Lasso: RMSE = 0.750
Trener Ridge...
Ridge: RMSE = 0.740
Beste modellen er SVR med en RMSE = 0.727
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

Som terminalen sier er den beste modellen SVR med et avvik RMSE = 0,727 avrundet.

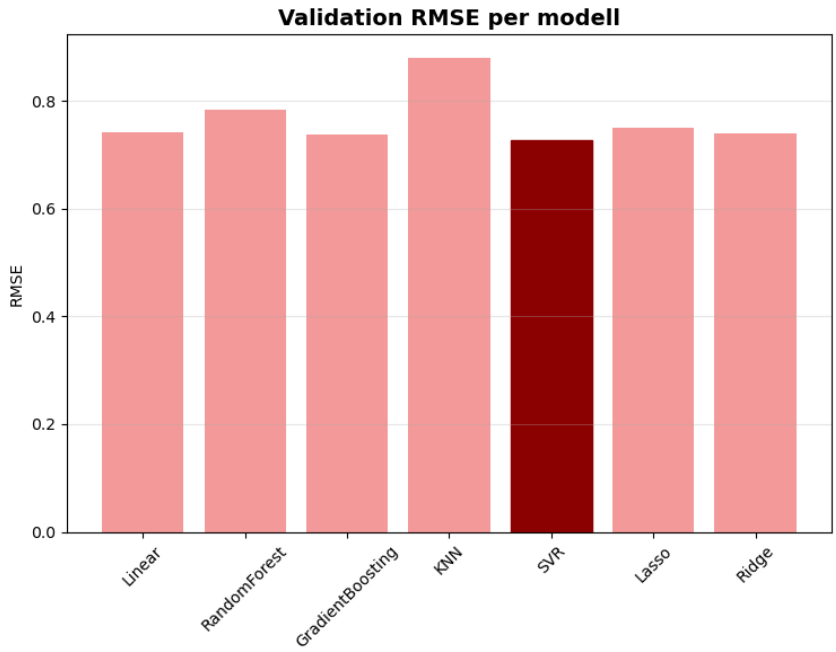
3.3 Sjekker generaliseringsevnen

En annen ting som er viktig å sjekke når vi skal finne den beste modellen, er generaliseringsevnen. Altså hvor godt modellen klarer seg på nye data. Dette tester jeg ved å sjekke validerings RMSE mot trenings RMSE. Vi vil ha disse to tallene mest mulig like, selv om begge skal være lave. En for lav train-RMSE kan tyde på overfitting, altså at modellen er for tilpasset treningsdataen. Jeg sjekker differansen ved å ta val_rmse – train_rmse, som kan ses i «train.py». Resultatene ble seende slik ut:

```
Linear:(Diff: -0.853)
RandomForest:(Diff: 0.204)
GradientBoosting:(Diff: -0.740)
KNN:(Diff: -0.608)
SVR:(Diff: -0.841)
Lasso:(Diff: -0.864)
Ridge:(Diff: -0.853)
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

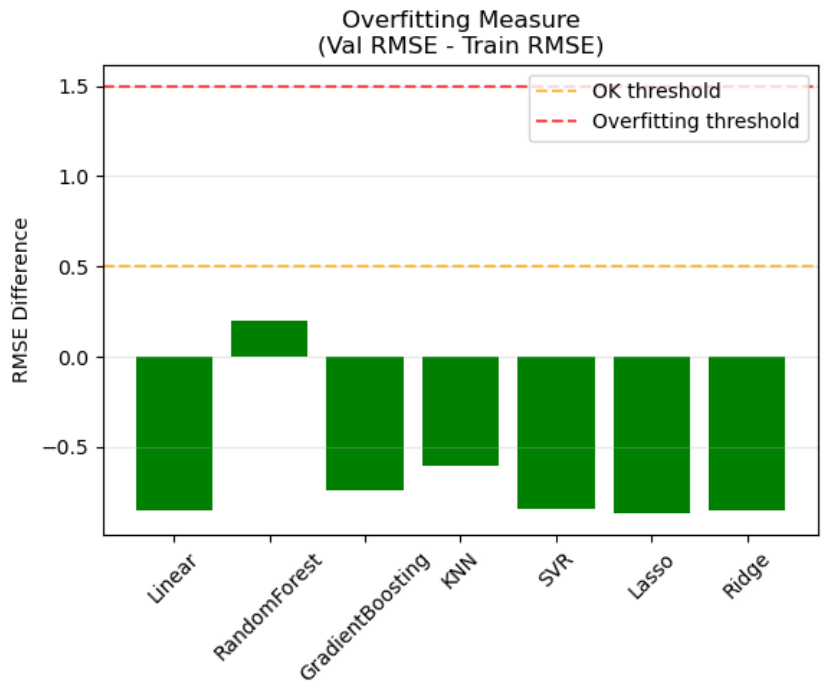
Som vi kan se har alle relativt god generaliseringsevne, og jeg velger å bruke SVR som min modell.

3.4 Visualiseringer av modellene



Figur 3.1, stolpediagram av RMSE til modellene

Som vi kan se leverer alle modellene generelt godt mot valideringsdataen.



Figur 3.2, RMSE differanse mellom validerings, og treningsdata, stolpediagram

3.5 Tester ulike hyperparametere av KNN

For å prøve å sikre den beste modellen kan det være lurt å sjekke ulike hyperparametere til modellene. KNN-modellen er en interessant modell der parameteren sjekker de n nærmeste naboene og ser likhet. Derfor prøver jeg KNN-modellen igjen med ulike n-verdier.

```
n=1: Train=0.000, Val=1.190, Diff=1.190
n=3: Train=1.225, Val=0.961, Diff=-0.264
n=5: Train=1.368, Val=0.908, Diff=-0.460
n=7: Train=1.431, Val=0.891, Diff=-0.541
n=10: Train=1.486, Val=0.879, Diff=-0.608
n=15: Train=1.536, Val=0.879, Diff=-0.658
n=20: Train=1.565, Val=0.878, Diff=-0.687
n=25: Train=1.587, Val=0.884, Diff=-0.703
n=30: Train=1.603, Val=0.888, Diff=-0.715
```

Som vi kan se i terminalskriften gir ikke ulike verdier av n, noe bedre RMSE, så jeg forblir på samme modell.

3.6 Prediksjon på test-data

For et endelig resultat av nøyaktigheten til modellen min sjekker jeg nøyaktigheten mot test-dataen. I koden min henter jeg modellen som den med lavest score blant resultatene. Trener på treningsdata som vanlig og predikerer y mot test. Det endelige testresultatet ble seende slik ut:

```
Final test RMSE: 1.534
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

Som vi ser fikk jeg en dårligere score mot test-dataen enn valideringsdataen. Dette kan det være flere grunner til, spesielt når jeg ikke vet hvordan test-dataen ser ut. Det kan ha vært et nytt data-hull, eller andre mønstre på dataen.

3.7 Prediksjons-resultat

For å vise hva modellen min faktisk predikerer har jeg laget en predict.py som oppgaven ber om. Denne tar inn rådata som train.py, og behandler dataen på relativt lik linje. Men her finnes det ingen trening av modeller, den bruker kun den lagrede beste modellen fra train.py, for prediksjonen.

```
Siste tidsstempel i data: 2025-05-02 17:49:25+02:00
Neste hele klokketime: 2025-05-02 18:00:00+02:00
Predikerer for tidsstempel: 2025-05-02 19:00:00+02:00

------------------------------------------------------------
Stasjon                    Nåværende sykler Predikerte sykler
------------------------------------------------------------
Akvariet                                  1                1
Damsgårdsveien 71                        13               13
Dreggsallmenningen Sør                   17               17
Florida Bybanestopp                      21               21
Grieghallen                              16               16
Høyteknologisenteret                     24               24
Møllendalsplass                           4                4
Studentboligene                           8                8
Torgallmenningen                         20               20
------------------------------------------------------------
PS C:\Users\brage\INF161\inf161-h25\prosjekt>
```

Her er endelig prediksjon av rådataen. Som terminal-utskriften viser har den faktisk predikert akkurat samme mengde ledige sykler neste time som i forrige observasjon. Dette er realistisk etter som modellen bruker et LOCF-prinsipp og modellen har lært at beste estimat for neste time er samme som nåværende.

4. Konklusjon

4.1 Forbedringer

En forbedring ville vært å bruke trips.csv bedre. Dette er det største datasettet med tanke på tidshorisonten. Om jeg hadde klart å utnytte det hadde jeg f.eks. hatt mer data å trene modellene mine på. I tillegg kunne kanskje mønstre på reisene utenfor mine stasjoner hjulpet modellen min å predikere mer nøyaktig, om jeg hadde fått til å utnytte det.

En mulig forbedring er å visualisere dataen tidligere, før jeg samlet den. Da hadde jeg kanskje sluppet å finne slike tids-hull som jeg gjorde.

Kunne håndtert manglende data fra «weather.csv» på en bedre måte. Brukte totalt snitt som gjør at den svinger mye.

Etter som datasettene er forskjellige tidsperioder blir splittingen før data-visualisering feil i forhold til etter alla dataen er merget sammen. Dette skaper lekasje. Dette var et problem som oppsto

For å prøve å finne en enda mer nøyaktig modell kunne jeg testet flere av modellene med ulike hyperparametere, for å optimere modellvalget.

4.2 Utfordringer underveis

Etter som datasettene er forskjellige tidsperioder blir splittingen før data-visualisering feil i forhold til etter alla dataen er merget sammen. Dette skaper lekkasje. Dette var et problem som oppsto. Jeg løste det ved å finne en felles overlap mellom all dataen.

Fikk først feilmelding ved trening av LinearRegression siden den ikke håndterer NaN-verdier. Oppdaget at de første radene inneholder NaN i fb_prev_h kolonnen, noe som gir mening siden "forrige times" verdi ikke eksisterer for første observasjon per stasjon. Valgte å fjerne disse radene.

4.3 Refleksjon

Hele prosjektet har gitt en veldig god trening i hele Data Science prosessen fra start til slutt. Veldig nyttig å dele opp prosjektet i flere deler, for og så ha en fagfellevurdering. Jeg lærte mye av å få tilbakemeldinger, men også av å få et innblikk i de andre sin kode, der jeg har hentet mye inspirasjon til smarte løsninger. I tillegg har det vært nyttig å holde på med et større prosjekt over en lengre periode, etter som jeg har lært mer lo lenger jeg har hatt faget. Jeg har blitt nødt til å endre på ting som jeg gjorde i del 1 og 2 som ikke har vært helt etter den riktige Data Science-protokollen.

Den største faktoren som hindrer meg fra å lage en bedre modell er nok mangelen på data. Det var god bredde i type features, men dataen skulle gjerne strukket seg over en mye lenger periode. Med enda lengre tid på selve prosjektet kunne jeg nok testet enda flere modeller, hyperparameter og hatt en bedre feature engineering. En modell som har en gjennomsnittlig kvadrert feil på 1,5 når den bare skal predikere sykler en time frem er ikke spesielt imponerende. Av prediksjonsresultatet ser det klart ut til at free_bikes og fb_prev_h har sterkest korrelasjon siden den predikerte helt lik mengde ledige sykler på stasjonene, mens features som vær ser ut til å ha liten korrelasjon, noe som vi også så i visualiseringen (figur 1.8).

Kilder

- Værdata fra Open meteo (https://open-meteo.com/) publisert under CC BY 4.0
- Bergen bysykkel data (https://bergenbysykkel.no/en/open-data) publisert under Norsk lisens for offentlige data (NLOD) 2.0 (https://data.norge.no/nlod/no/2.0)
- https://github.com/MaxHalford/bike-sharing-history
- Discord server INF161

Bruk av KI

Jeg har brukt chatGPT.com som sparringspartner. Jeg har ikke kopiert noe kode, men brukt KI mest til forståelse og forklaring.